1. DATA EXPLORATION AND PREPROCESSING

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_excel('/content/Customer_data.xlsx')
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneServi
0	7590-VHVEG	Female	0	Yes	No	1	
1	5575-GNVDE	Male	0	No	No	34	`
2	3668-QPYBK	Male	0	No	No	2	`
3	7795-CFOCW	Male	0	No	No	45	
4	9237-HQITU	Female	0	No	No	2	`

5 rows × 21 columns

```
df.shape
```

```
(7043, 21)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   customerID        7043 non-null   object 
 1   gender            7043 non-null   object 
 2   SeniorCitizen     7043 non-null   int64  
 3   Partner           7043 non-null   object 
 4   Dependents        7043 non-null   object 
 5   tenure            7043 non-null   int64  
 6   PhoneService      7043 non-null   object 
 7   MultipleLines     7043 non-null   object 
 8   InternetService   7043 non-null   object 
 9   OnlineSecurity    7043 non-null   object 
 10  OnlineBackup      7043 non-null   object 
 11  DeviceProtection  7043 non-null   object 
 12  TechSupport       7043 non-null   object 
 13  StreamingTV       7043 non-null   object 
 14  StreamingMovies   7043 non-null   object 
 15  Contract          7043 non-null   object 
 16  PaperlessBilling  7043 non-null   object 
 17  PaymentMethod     7043 non-null   object 
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7032 non-null   float64
 20  Churn             7043 non-null   object 
dtypes: float64(2), int64(2), object(17)
memory usage: 1.1+ MB
```

```
df.describe()
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7043.000000	7043.000000	7043.000000	7032.000000
mean	0.162147	32.371149	64.761692	2283.300441
std	0.368612	24.559481	30.090047	2266.771362
min	0.000000	0.000000	18.250000	18.800000
25%	0.000000	9.000000	35.500000	401.450000
50%	0.000000	29.000000	70.350000	1397.475000
75%	0.000000	55.000000	89.850000	3794.737500
max	1.000000	72.000000	118.750000	8684.800000

```
df.isnull().sum()
```

	0
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	11
Churn	0

dtype: int64

```
df.nunique()
```

	0
customerID	7043
gender	2
SeniorCitizen	2
Partner	2
Dependents	2
tenure	73
PhoneService	2
MultipleLines	3
InternetService	3
OnlineSecurity	3
OnlineBackup	3
DeviceProtection	3
TechSupport	3
StreamingTV	3
StreamingMovies	3
Contract	3
PaperlessBilling	2
PaymentMethod	4
MonthlyCharges	1585
TotalCharges	6530
Churn	2

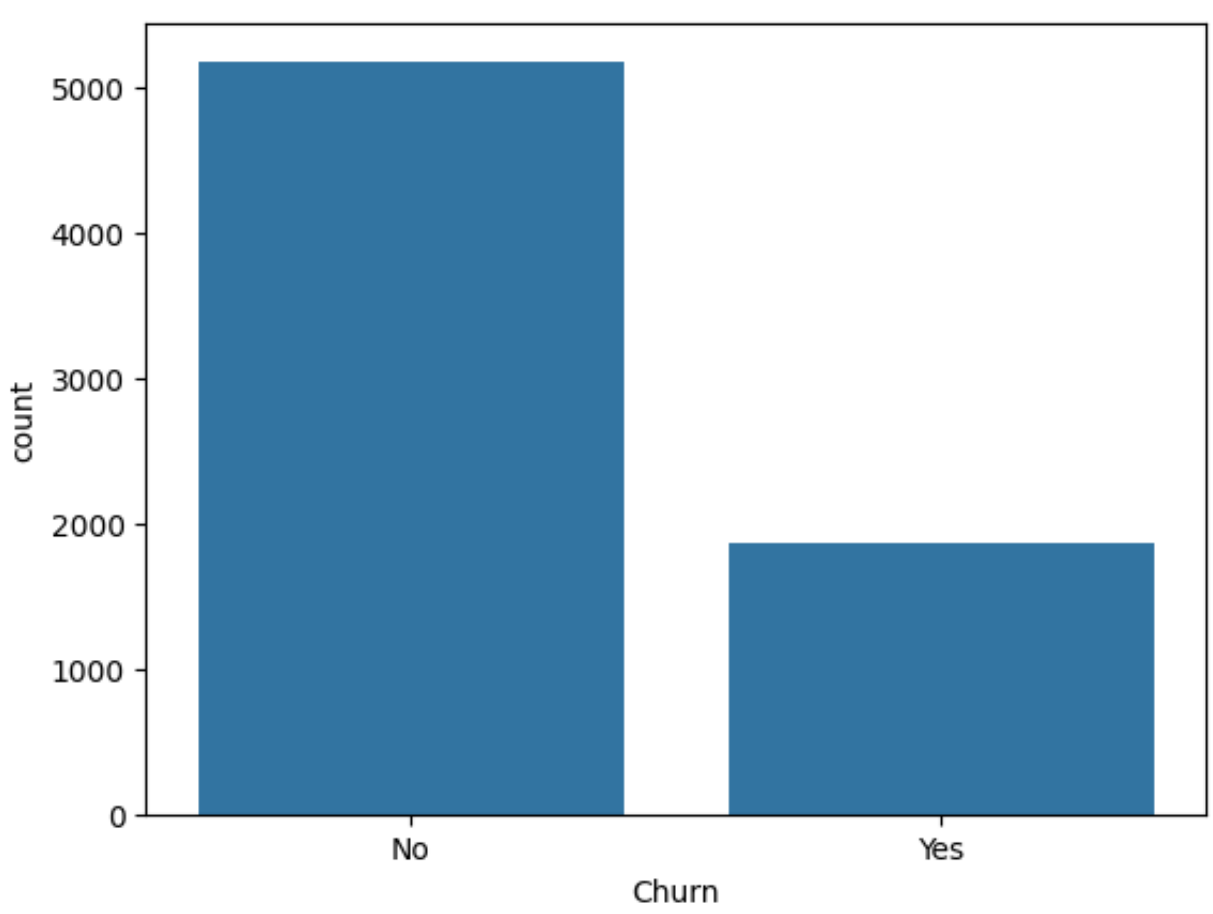
dtype: int64

```
df['Churn'].value_counts()
```

Churn	count
No	5174
Yes	1869

dtype: int64

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='Churn', data=df)
plt.show()
```



```
df = df.drop(columns=['customerID'])
df.head(2)
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Multiple
0	Female	0	Yes	No	1	No	No phone
1	Male	0	No	No	34	Yes	

```
df = df.replace(" ", np.nan)
df = df.dropna()
df.isnull().sum()
```

	0
SeniorCitizen	0
tenure	0
MonthlyCharges	0
TotalCharges	0
gender_Male	0
Partner_Yes	0
Dependents_Yes	0
PhoneService_Yes	0
MultipleLines_No phone service	0
MultipleLines_Yes	0
InternetService_Fiber optic	0
InternetService_No	0
OnlineSecurity_No internet service	0
OnlineSecurity_Yes	0
OnlineBackup_No internet service	0
OnlineBackup_Yes	0
DeviceProtection_No internet service	0
DeviceProtection_Yes	0
TechSupport_No internet service	0

TechSupport_Yes	0
StreamingTV_No internet service	0
StreamingTV_Yes	0
StreamingMovies_No internet service	0
StreamingMovies_Yes	0
Contract_One year	0
Contract_Two year	0
PaperlessBilling_Yes	0
PaymentMethod_Credit card (automatic)	0
PaymentMethod_Electronic check	0
PaymentMethod_Mailed check	0
Churn_Yes	0

dtype: int64

```
df['TotalCharges'] = df['TotalCharges'].astype(float)
df[['tenure', 'MonthlyCharges', 'TotalCharges']].describe()
```

	tenure	MonthlyCharges	TotalCharges
count	7032.000000	7032.000000	7032.000000
mean	32.421786	64.798208	2283.300441
std	24.545260	30.085974	2266.771362
min	1.000000	18.250000	18.800000
25%	9.000000	35.587500	401.450000
50%	29.000000	70.350000	1397.475000
75%	55.000000	89.862500	3794.737500
max	72.000000	118.750000	8684.800000

```
df = pd.get_dummies(df, drop_first=True)
df.shape
```

```
(7032, 31)
```

```
X = df.drop('Churn_Yes', axis=1)
y = df['Churn_Yes']
y.value_counts()
```

Churn_Yes	count
False	5163
True	1869

dtype: int64

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random
X_train.shape, X_test.shape
```

```
((5625, 30), (1407, 30))
```

2. MODEL DEVELOPMENT

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train_scaled, y_train)

print("Test Accuracy:", model.score(X_test_scaled, y_test))
```

```
Test Accuracy: 0.7874911158493249
```

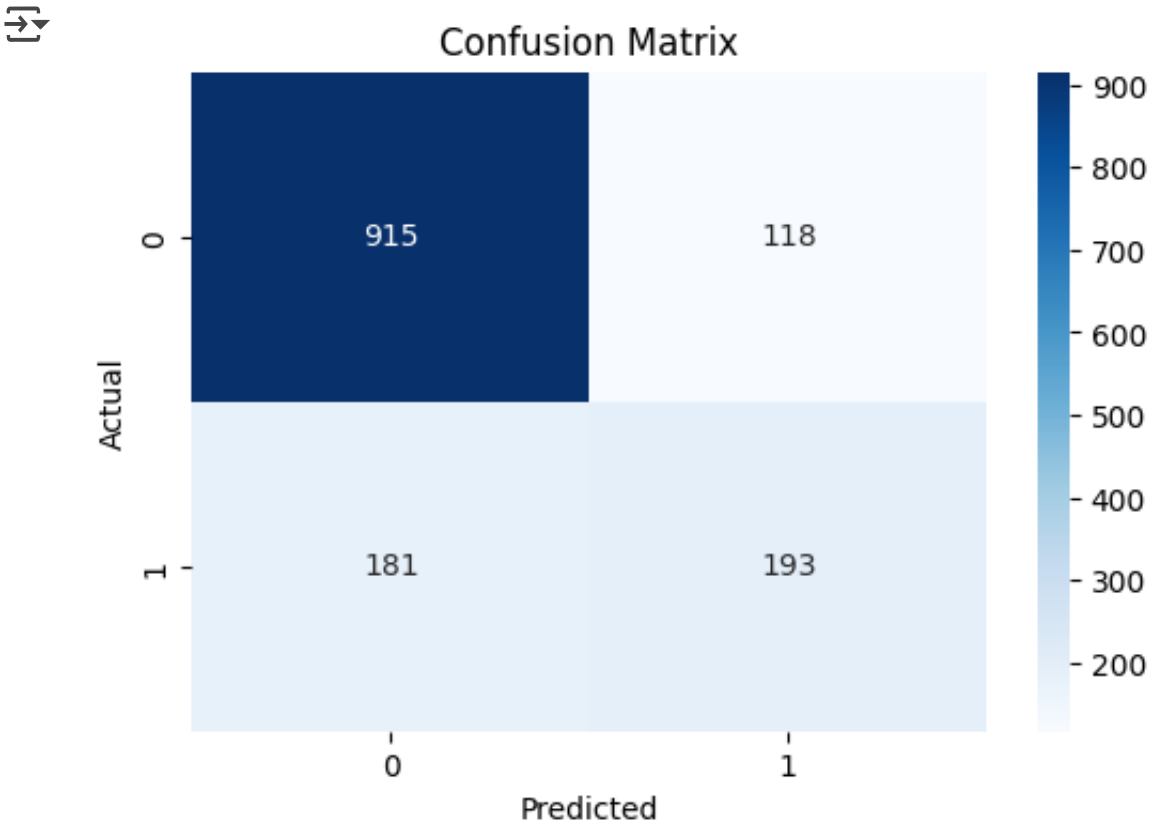
3. MODEL EVALUATION

```
from sklearn.metrics import classification_report

y_pred = model.predict(X_test_scaled)
print(classification_report(y_test, y_pred, digits=3))
```

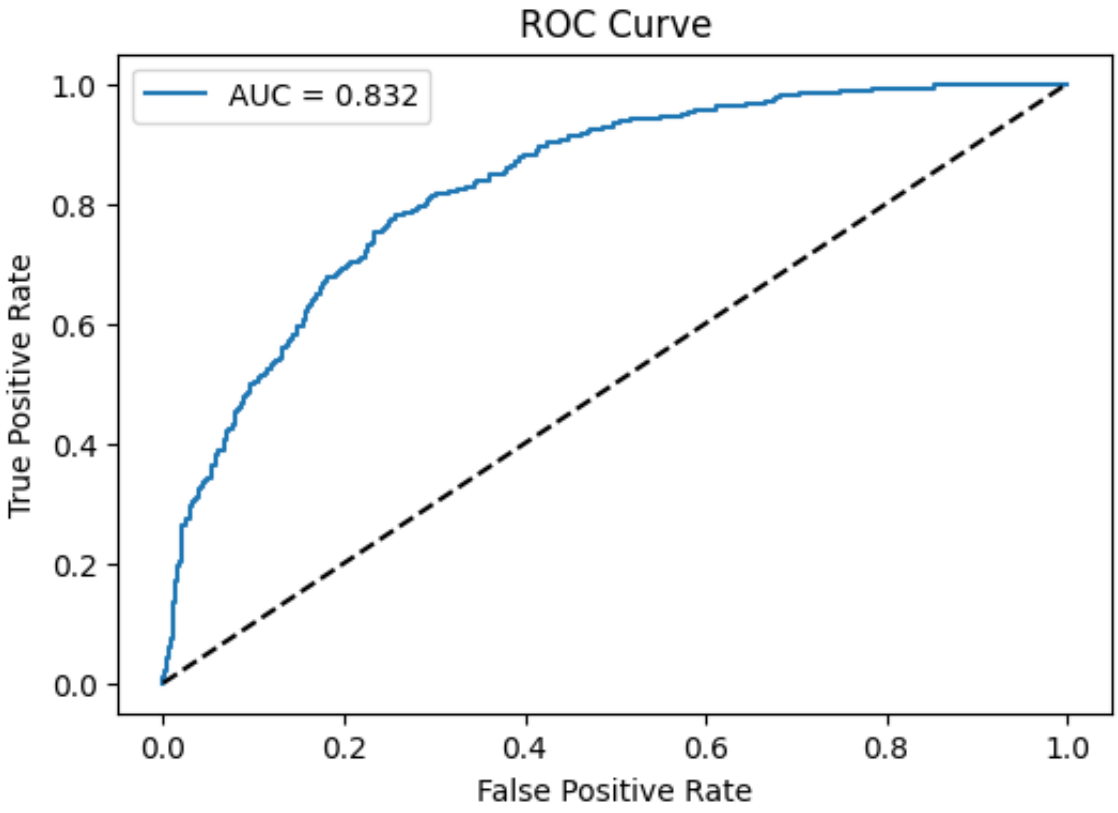
```
              precision    recall  f1-score   support

       False      0.835     0.886     0.860      1033
        True      0.621     0.516     0.564       374

    accuracy                          0.787      1407
   macro avg      0.728     0.701     0.712      1407
weighted avg      0.778     0.787     0.781      1407
```

4. FINAL VISUALIZATION

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Generate confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Plot the heatmap
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



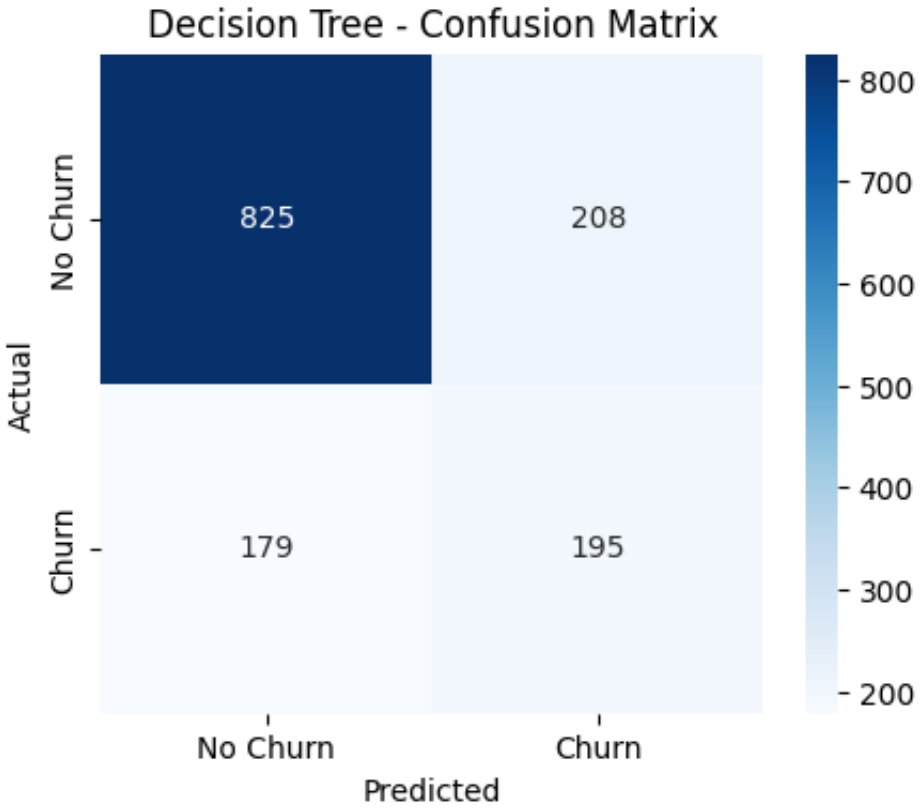
```
from sklearn.metrics import roc_curve, roc_auc_score

# Predict probabilities
y_probs = model.predict_proba(X_test_scaled)[:, 1]

# Compute ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_probs)

# Plot ROC curve
plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc_score(y_test, y_probs):.3f}")
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```



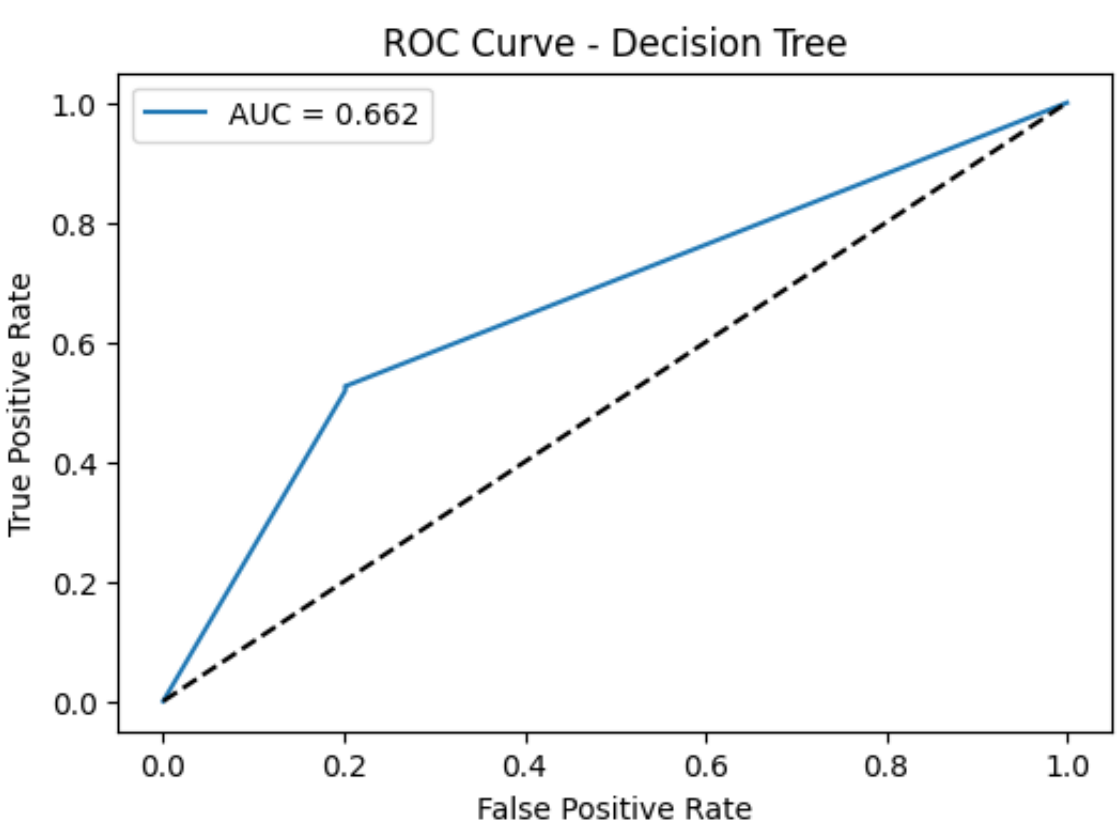
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report

# Train Decision Tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_dt = dt_model.predict(X_test)
print("Decision Tree Classification Report:\n")
print(classification_report(y_test, y_pred_dt, digits=3))
```

```
Decision Tree Classification Report:

              precision    recall  f1-score   support

       False      0.822     0.799     0.810      1033
        True      0.484     0.521     0.502       374

    accuracy                          0.725      1407
   macro avg      0.653     0.660     0.656      1407
weighted avg      0.732     0.725     0.728      1407
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_dt)

# Plotting
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No Churn', 'Ch
plt.title('Decision Tree - Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



```
from sklearn.metrics import roc_curve, roc_auc_score

# Get probability scores for positive class
y_probs_dt = dt_model.predict_proba(X_test)[:, 1]

# ROC values
fpr_dt, tpr_dt, thresholds_dt = roc_curve(y_test, y_probs_dt)
auc_dt = roc_auc_score(y_test, y_probs_dt)

# Plot ROC
plt.figure(figsize=(6,4))
plt.plot(fpr_dt, tpr_dt, label=f"AUC = {auc_dt:.3f}")
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Decision Tree")
plt.legend()
plt.show()
```



```
logistic_acc = model.score(X_test_scaled, y_test)
logistic_auc = roc_auc_score(y_test, y_probs)

tree_acc = dt_model.score(X_test, y_test)
tree_auc = roc_auc_score(y_test, y_probs_dt)

print("Logistic Regression - Accuracy:", round(logistic_acc, 3), "| AUC:", rour
print("Decision Tree       - Accuracy:", round(tree_acc, 3), "| AUC:", round(tr
```

```
Logistic Regression - Accuracy: 0.787 | AUC: 0.832
Decision Tree       - Accuracy: 0.725 | AUC: 0.662
```

Google Drive Link to project Video Explanation, Presentation , and Code Files:

https://drive.google.com/drive/folders/1vymZjqQsC_arDYKrcLXiFrZPPCNh-b46?usp=sharing